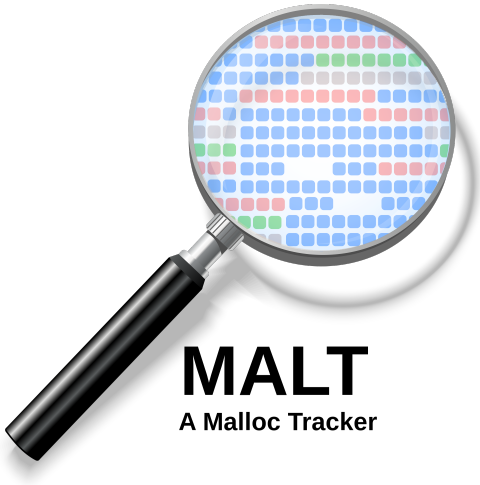
# MALT

*Release 1.4.1*

**Sébastien Valat**

**Dec 11, 2025**

# GETTING STARTED


MALT
A Malloc Tracker

CHAPTER

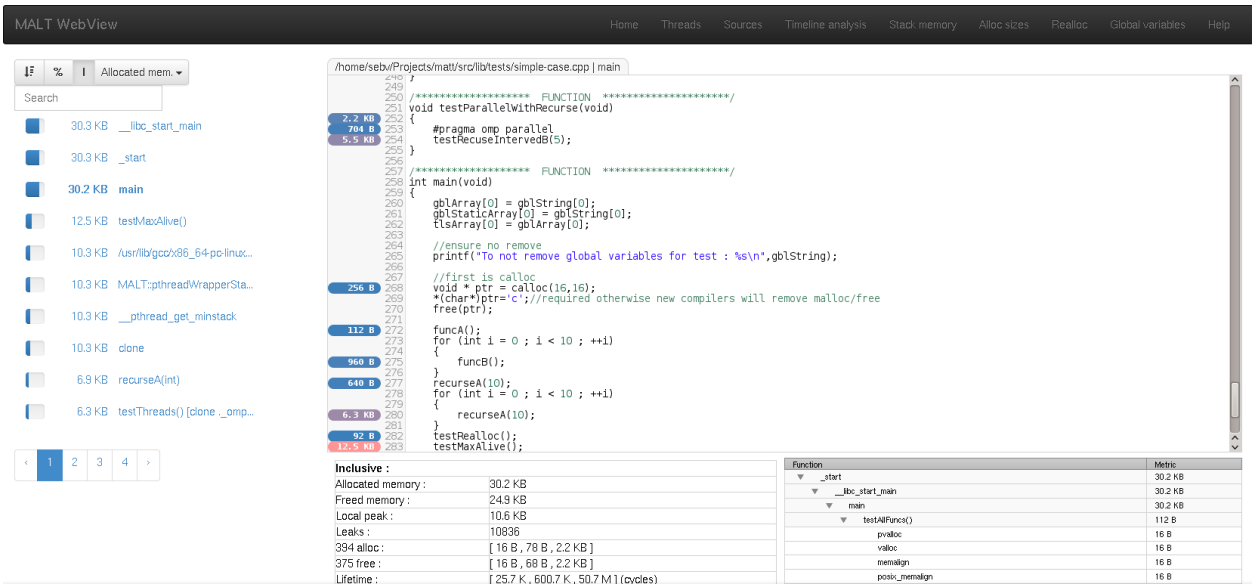
# ONE

# WHAT IS MALT ?

MALT is a memory tool to find where you allocate your memory. It also provides you some statistics about memory usage and help to find memory leaks.

It is done to be used on laguages : **C**, **C++**, **Fortran**, **Rust**, **Python** *(expérimental)*.

It comes with a web-based graphical interface to navigate the profile :



CHAPTER

TWO

# FIRST TIME HERE?

We have a few places for you to get started:

***Dependencies***
: Start by installing the **required** or **optional** dependencies.

***Installation***
: Follow the **installation procedure**.

***Basic usage***
: Make your first steps by **using malt**.

***Understanding the metrics***
: The metrics exported by **malt**.

***Informations***
: General informations about **malt**.

CHAPTER

THREE

# ADVANCED TOPICS

*MPI*
: Using MALT for **MPI** applications.

*Using custom allocator*
: Using MALT with **custom memory allocators**.

*Sub-part of a program*
: Using MALT on a **sub-part of your programe**.

*Using -finstrument-functions*
: Using MALT with **-finstrument-functions**.

*Track out of memory*
: Using MALT to track **out of memory** on a program.

*Options*
: Using MALT **options**.

*Build options*
: Compiling MALT with extra **build options**.

*Packaging for distributions*
: Building **packaging** for common distributions.

CHAPTER

FOUR

# EXTERNAL POINTERS

*Other tools*
: Discovering some other **alternative tools**.

*Other memory allocators*
: Discovering some **alternative memory allocators**.

*Documentation about memory*
: Documentation about **memory managment**.

## 4.1 Dependencies

Here are the dependencies for **building** from source and **running** MALT.

### 4.1.1 Required dependencies

The dependencies of MALT are the following if you use the **official releases** which embed some extra files compared to the **GIT repository**:

**Binutils**

- 2.24 - 2.38
- Required for the **nm** and **addr2line** tools to solve the symbols and get the global variables informations.

**CMake**

- 2.8 - 3.28.3
- Required to **build** MALT from source.

### 4.1.2 Optional dependencies

Those dependencies are not required but will permit to **enable some features** support of MALT.

**libelf**

- 0.128 - 0.183
- To extract **global variables** list from executables and libraries.

**libunwind**

- 3.11 - 3.13
- An alternative implementation of glibc **backtrace** method

**libpython**

- 3.11 - 3.14

- To activate the support of **Python** language in MALT.

**Graphviz**

- 2.42.2
- To activate the **call graph rendering** in the webview via the **dot** command.

### 4.1.3 Embeded dependencies

Some dependencies are **embeded** into the GIT repository to ease installation of MALT because most of the time not present by default on available systems. **If present** MALT will use the **system version**.

**httplib**

- 0.23.1
- To implement the C++ based **webview server** REST API.

**GoogleTest**

- 1.14.0
- To build and run the MALT **unit tests**.

**Nlohmann-json**

- 3.12.0
- Use to decode / encoder the **JSON** data in the **webview**.

### 4.1.4 NodeJS dependencies

The **webview** is based on the *VueJS* framework, so it requires some dependencies for **NodeJS** in order to be built.

The CMake script automatically **download** them if you are using the **git** repository as it does not contains those dependencies.

The **official release archive** embed them and contains a pre-build version of the webview so you do not requires **NodeJS / NPM** nor the **node_modules** to get it running on your system.

### 4.1.5 Dependencies for the GIT repo

If you want to use the **GIT repository** you might also need some extra dependences to get some files not embeded in not embeded into the **GIT repository** but present in the **officiel release archive** :

**NodeJS**

- 0.10.30 - 12.22.9
- Required if you want to download the **interface dependences** via **npm** which are packed into the officiel release.

**Curl**

- 8.5.0
- Required to **download** the sources of **JeMalloc** memory allocator. Those sources are packed in the the official release.

### 4.1.6 Dependencies in distributions

If you want to install the dependencies provided by your system you can easily use the following commands.

#### Debian / Ubuntu

If you are using an **APT** based distribution derivated from **Debian** you can use :

```
sudo apt install cmake g++ make libssl-dev libunwind-dev \
                 libelf-dev libunwind-dev nodejs npm \
                 nlohmann-json3-dev graphviz python3-dev curl
```

#### Fedora / Rocky

If you are using an **RPM** based distribution derivated from **Fedora** you can use :

```
sudo dnf install -y cmake gcc-c++ make \
                    openssl-devel elfutils-libelf-devel \
                    nodejs npm curl bzip2 xz graphviz python3-devel
```

#### Archlinux

If you are using **Archlinux** :

```
pacman -Sy cmake gcc make openssl \
           libunwind libelf nodejs npm curl \
           nlohmann-json graphviz python3
```

#### Gentoo

If you are using **Gentoo** :

```
emerge cmake gcc make openssl sys-libs/libunwind elfutils \
       nodejs curl dev-cpp/nlohmann_json media-gfx/graphviz
```

## 4.2 Installation

### 4.2.1 Compiling

To **compile** MALT, simply use :

If you are using **Gentoo** :

```
mkdir build
cd build
../configure --prefix=$HOME/usr-malt
make -j8
make install
```

Note that the **configure** script is just a **wrapper** with the **configure** classic semantic on top of **cmake**, so you can also call directly **cmake**:

```
mkdir build
cd build
cmake -DCMAKE_INSTALL_PREFIX=$HOME/usr-malt -DCMAKE_BUILD_TYPE=Release
make -j8
make install
```

### 4.2.2 Note about Intel Compiler

MALT is written in C++ so you might possibly encounterd some issue with you build it with GCC and profile applications built with Intel Compiler. In most cases it should work out of the box without any issues.

But, I got once an error report about that. In that case, try to compile MALT also with intel compiler instead if GCC to match the app :

```
# with the configure script
../configure CC=icc CXX=icpc
make

# with the cmake script directly
cmake -DCMAKE_C_COMPILER=icc -DCMAKE_CXX_COMPILER=icpc
```

### 4.2.3 Installation in non-standard directory

If you install MALT in a directory other than */usr* and */usr/local*, eg. in your home, you might be interested by setting some environment variables integrating it to your shell :

```
export PATH=${PREFIX}/bin:$PATH
export MANPATH=${PREFIX}/share/man:$MANPATH
```

*LD_LIBRARY_PATH* is not required as the *malt* command will use the full path to get access the internal *.so* file.

### 4.2.4 Installing via spack

If you are using Spack you can simply use it to build the **dependencies** and **MALT** :

```
./bin/spack install malt
```

### 4.2.5 Installing via Gentoo Overlay

On Gentoo you can also directly install MALT by using the given **overleay** :

```
sudo eselect repository add memtt git https://github.com/memtt/gentoo-memtt-overlay.git
sudo eselect repository enable memtt
sudo emerge -a malt
```

### 4.2.6 Building distribution packages

MALT provide the files necessary to produce the **APT** and **RPM** packages, you will find the procedure to use them in the *advanced section*.

## 4.3 Basic usage

Now you have installed **MALT** you can proceed by using it.

### 4.3.1 Examples directory

In order to start playing with **MALT** you can find some examples into the directory */PREFIX/share/malt/examples*.

### 4.3.2 Profiling a C application

If you have a C / C++ / Fortran / Rust application, first compile it by using the *-g* debug option to be able to find the source lines from the binaries.

```
# Compile with debug option -g
gcc -g -O3 -o basic-example ./main.c
```

The simply run within **malt** :

```
# Run by wrapping your command with 'malt'
malt ./basic-example
```

It will produce a file named *malt-basic-example-PID.json* to be analised offline.

### 4.3.3 Browsing the profile

You can browser into the profile by using the command *malt-webview*. This command starts a lightweight server locally so you can connect on it with your web browser onto http://localhost:8080 by default.

Note that it will ask you for a token which is displayed into the terminarl when the mini-server is ready to render the pages.

```
# run the mini server
malt-webview ./malt-basic-example-5656.json
# open your browser on http://localhost:8080.
```

### 4.3.4 Browsing remotely

MALT has been developped for **HPC clusters**, in this case you might have made your profiling **run** on a **remote cluster** and want to visualize the output on your **localhost workstation**.

In this case proceed by using the **port forwarding** feature of the *ssh* command.

The more secure approch as supported by recent version of *ssh* is by using a local **socket file**:

```
# ssh with port forwaring
ssh -L 8080:localhost:~/.malt-socket.1 user@server

# launch the webview as normal
malt-webview --port=~/.malt-socket.1 ./malt-basic-example-5656.json
```

You can either use a **port forwarding** but it opens to other people on the server and needs to select **another port** if you are many using MALT on the server.

```
# ssh with port forwaring
ssh -L 8080:localhost:8080 user@server
```

(continues on next page)

(continued from previous page)

```
# launch the webview as normal
malt-webview --port 8080 ./malt-basic-example-5656.json
```

### 4.3.5 Fast profiling

If you want to make a fast profiling without extracting the call stacks (which adds a large overhead by the tool), you can disable the stacks by using :

```
# Run fast without analysing the stacks
malt -o stacks:mode=none ./basic-example
```

It will produce the profile with all the counters except the call tree and source annotations. By the way it has in this mode a very low overhead to get a first view on the dynamic of the application.

### 4.3.6 Profiling python only

Since release **1.4.0** malt supports profiling Python scripts. To proceed, simply use the *malt-python* command instead of *malt*.

```
# profiling python only
malt-python ./my-script.py

# read the profile with the webview
malt-webview ./malt-python-my-script-5656.json
```

As for the C case, it will produce a file *malt-python-my-script-PID.json* which you open via the web interface.

Under python the overhead is very hight, if you want to get a first look on the global metrics, first make a run with the **no-stack** profile.

```
# profiling python only
malt-python --profile python-no-stack ./my-script.py
```

### 4.3.7 Profiling python and C

This kind of analysis increase the overhead so it is not enabled by default on the *malt-python* command. In order to proceed, use the *–profile* option :

```
# see both domains splitted
malt-python --profile python-domains

# see both domains merged as a single call tree
malt-python --profile python-mix
```

## 4.4 Understanding the metrics

MALT exposes many metrics over its profiles, all the metrics will be listed and explained here.

### 4.4.1 Reminder on Memory

**MALT** as it is named (MAlloc Tracker) aimed at tracking all the *malloc()* of an application to report the memory used by itself.

For a C, C++, Rust, Fortran application there is 4 ways to request memory to the system from the language semantic.

You can find the general view here :

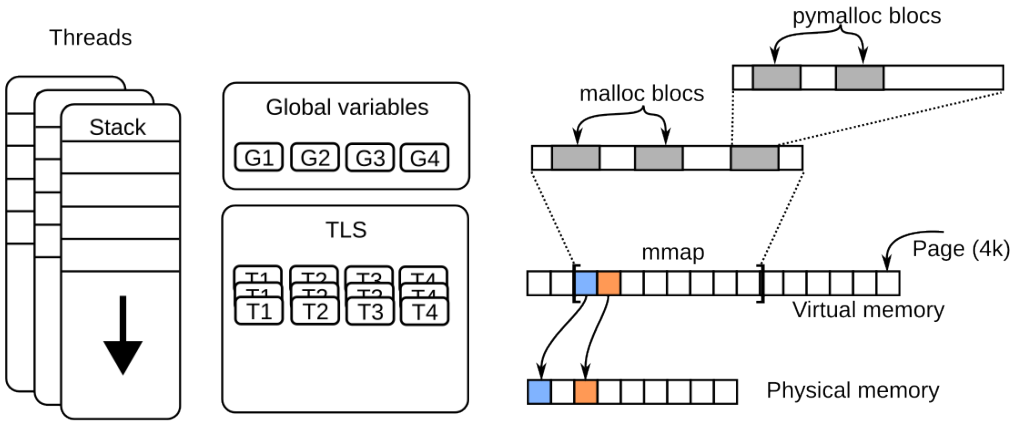



Fig. 1: The 4 memory consumption parts.

#### Global variables

The first common way is the **global variables**, they are allocated globaly for the run when spawning the process. Except if it contains specific values at start, a global variable is initially a **virtual** space allowed to be accessed in the memory and filled by default with **zeroes**.

Note that is you do not access it it will use **virtual memory** but not **physical one** You will find in MALT the ratio between those two consumption about global variables.

The amount of **virtual memory** is fixed at the start of the app, but the **physical memory** will increase over the run as we access the variable making it **mapped in memory**.

This is why you can see in the time charts in **increase** of the **physicial memory** without making any calls to mallocs.

#### Thread Local Storage (TLS)

The **global variables** are shared between all threads. In C, C++ there is a way to use an annotation to make a global variable **private to each thread**. This is called a **Thread Local Storage** says in acronym, a **TLS**. In this case, the memory consumption of the variable will be the **size** of the varaible times the **number of threads**.

#### Stacks

The **local variables** inside **functions** are stored into the stacks which can grow as we go deeper in the call hierarchy mainly when we do recursion. MALT is currently unperfect to track this memory but tries to give some hints if you compiled you code with *-finstrument-functions* on GCC compiler.

#### mmap

This is a call you generally never use at an app programer level. *mmap* is the main function to request memory to the operating system. It by default provides a segment which is virtually mapped. Meaning that is consumes **virtual memory** but not **physical memory**. By default on a first access, (**first touch** we say) a **physical page** will be **filled with zeroes** and mapped into the segment.

Notice here that the **zeroes filling** can imply a large overhead when the segment is large. This is way we should keep large allocations for a long time and not allocate / deallocate them intensively otherwise we can pass half of our time to just write zeroes into the memory.

Note that the cost of the **zeroing** will not be **paied** at the **allocation time** but latter in the computation **at the first access** to the considered memory region.

As a by design **restriction**, *mapp* only allow managing **sizes** and **addresses** multiple of then **hardware page size** generally **4 KB**. This is way it is rarely used directly the application codes.

Remark that MALT will account the *mmap* calls only if they are done by the user, it will not count the once made by the **memory allocator** itself.

#### malloc

As *mmap* do not allow to handle any kind of sizes, *malloc()* has been built to fill this gap and by itself call *mmap* and handle the splitting of the segments it handles to store the allocations you request **about any kind of sizes**.

This is in C what is used to allocate **dynamic memory** and is the main function tracked by **MALT**.

### 4.4.2 Metrics

Here a short summary of the matrics exported by MALT :

Table 1: Table of metrics

| Metric | Short description |
|---|---|
| Allocated mem | Sum of all the mallocs and mmap done. |
| Allocated count | Count of all calls to malloc() and mmap() |
| Min. alloc size | Minimum block size allocated at this location. |
| Max. alloc size | Maximum block size allocated at this location. |
| Freed mem. | Amount of memory freed on this line. |
| Free count. | Number of call to free() function on this line. |
| Memory ops. | Total number of memory ops done on this line (malloc + mmap + free). |
| Local peak | Peak for this local line. |
| Global peak | Contribution to the peak of this specific line. |
| Leaks | Amount of memory leaking on this line. |
| Max lifetime | Maximum lifetime of the blocks on this line. |
| Min lifetime | Minimum lifetime on this line. |
| Recycling ratio | Number of time this line reallocated its peak memory. |
| Realloc count | Number of calls to realloc(). |
| Realloc sum | Amount of memory allocated in total by realloc(). |
| Mmap mem. | Amount of memory allocated in total by mmap(). |
| Mmap count | Number of calls to mmap. |
| Munmap mem. | Amount of memory freed in total by munmap(). |
| Munmap count | Number of calls to munmap. |

#### Allocated memory

This metric just **sum up** the sizes given to all **mallocs** and **mmap**. In other words it represent the **memory volume** to be managed by the allocator and the system.

This metric give an indirect view on the time which is require to mange this volume of memory. Mainly if it **becomes gigatbytes**, it means this the code might take a long time to **exchange** its **memory** with the **operating system**.

Lets take an example:

```
void func() {
    void * a = malloc(32);          //allocated mem. of func = 32
    void * b = malloc(32);          //allocated mem. of func = 64
    free(b);
    void * c = malloc(64);          //allocated mem. of func = 128
    free(c);
    free(a);
    void * d = mmap(... 4096 ...);  //allocated mem. of func = 128 + 4096
    munmap(d...);
}

main() {
    func();                         //allocated mem. = 128 + 4096
}
```

### Allocated count

The numer of memory allocation for the given line. Accounting the **malloc** and **mmap**. It show the potential oberhead due to **malloc** itself.

```
void func() {
    void * a = malloc(32);          //allocated count of func  = 1
    void * b = malloc(32);          //allocated count of func  = 2
    free(b);
    void * c = malloc(64);          //allocated count of func  = 3
    free(c);
    free(a);
    void * d = mmap(... 4096 ...);  //allocated count of func  = 4
    munmap(d...);
}

main() {
    func();                         //allocated count = 4
}
```

### Min. alloc size

The minimum size passed as parameter of **malloc** or **mmap** on the given site. It permits to find the small memory allocations.

```
void func() {
    void * a = malloc(32);          //min. alloc size of func = 32
    void * b = malloc(32);          //min. alloc size of func = 32
    free(b);
    void * c = malloc(16);          //min. alloc size of func = 16
    free(c);
    free(a);
    void * d = mmap(... 4096 ...);  //min. alloc size of func = 16
    munmap(d...);
}

int main() {
```

(continues on next page)

(continued from previous page)

```
    func();                         //min. alloc size = 16
}
```

### Max. alloc size

Maximum size passed to **malloc** or **mmap** on the given line. It permits to find the large memory allocations in the application.

```
void func() {
    void * a = malloc(32);          //max. alloc size of func = 32
    void * b = malloc(32);          //max. alloc size of func = 32
    free(b);
    void * c = malloc(16);          //max. alloc size of func = 32
    free(c);
    free(a);
    void * d = mmap(... 4096 ...);  //max. alloc size of func = 4096
    munmap(d...);
}

int main() {
    func();                         //max. alloc size = 4096
}
```

### Freed mem.

Amount of memory freed on the given line by *free* or *munmap*. It is computed by summing the size of all the blocks freed on the given location.

```
void func() {
    void * a = malloc(32);
    void * b = malloc(32);
    free(b);                        //freed mem. of func = 32
    void * c = malloc(16);
    free(c);                        //freed mem. of func = 48
    free(a);                        //freed mem. of func = 80
    void * d = mmap(... 4096 ...);
    munmap(d...);                   //freed mem. of func = 80 + 4096
}

int main() {
    func();                         //freed count. = 80 + 4096
}
```

### Free count

Number of calls to **free** and **munmap** functions.

```
void func() {
    void * a = malloc(32);
    void * b = malloc(32);
    free(b);                        //freed count. of func = 1
    void * c = malloc(16);
```

(continues on next page)

(continued from previous page)

```
    free(c);                        //freed count. of func = 2
    free(a);                        //freed count. of func = 3
    void * d = mmap(... 4096 ...);
    munmap(d...);                   //freed count. of func = 4
}

int main() {
    func();                         //freed count. = 4
}
```

### Memory ops

Count the number of memory operations done on a given location.

```
void func() {
    void * a = malloc(32);          // memory ops. of func = 1
    void * b = malloc(32);          // memory ops. of func = 2
    free(b);                        // memory ops. of func = 3
    void * c = malloc(16);          // memory ops. of func = 4
    free(c);                        // memory ops. of func = 5
    free(a);                        // memory ops. of func = 6
    void * d = mmap(... 4096 ...);  // memory ops. of func = 7
    munmap(d...);                   // memory ops. of func = 8
}

int main() {
    func();                         // memory ops. of func = 8
}
```

### Local peak

This is the memory consumption peak for a given code location. It takes in account all the memory allocation done on this particular location.

```
void func() {
    void * a = malloc(32);          // local peak of func = 32
    void * b = malloc(32);          // local peak of func = 64
    free(b);
    void * c = malloc(16);          // local peak of func = 64
    free(c);
    void * d = mmap(... 4096 ...);  // local peak of func = 32 + 4096
    munmap(d...);
}

int main() {
    void * buffer = malloc(256);
    func();                         // local peak = 48 + 4096
}
```

### Global peak

This metrics gives the **memory consumption** of the local site **at the peak time**.

Notice it can arise at a **different moment** than the **local peak** because the global peak does not necesserily arise at the time the local peak arise.

### Leaks

This is the **mallocs** which are not paired with a **free** or an **mmap** not paried or partially paired with an **munmap**.

```
void func() {
    void * a = malloc(32);          // leak = 32
    void * b = malloc(32);
    free(b);
    void * c = malloc(16);
    free(c);
    void * d = mmap(... 4096 ...);
    munmap(d...);
}

int main() {
    void * buffer = malloc(256);
    func();                         // leak = 32
}
```

### Max lifetime

It takes the maximal **delay** between a **malloc** and its **paired free** on the given location.

If this value is too low and there is a large amount of allocation done at the given location it means that there is possibly a performance issue.

### Min lifetime

It takes the minimal **delay** between a **malloc** and its **paired free** on the given location.

### Recycling ratio

It compute the ratio of the **allocatd volume** over the **local peak**. It give a rougth estimation of the number of time the function reallocated its peak memory.

```
void func() {
    void * a = malloc(32);          // recyclong ratio = 1
    free(a);
    for (size_t i = 0 ; i < 1000 ; i++) {
        void * c = malloc(16);      // recyclong ratio = 1000
        free(c);
    }
}

int main() {
    void * buffer = malloc(256);
    func();                         // recyclong ratio = 501 ((32 + 16*1000) / 32)
}
```

### Realloc count

Count the number of *realloc* done at this location.

```
void func() {
    void * a = realloc(nullptr, 32);       // realloc count of func = 1
    a = realloc(a, 64);                    // realloc count of func = 2
    a = realloc(a, 16);                    // realloc count of func = 3
    free(a);
}

int main() {
    func();                                // realloc count of func = 3
}
```

### Realloc sum

Sum the memory delta of each *realloc* done at the given location.

```
void func() {
    void * b = realloc(nullptr, 10);       // realloc sum of func = 10
    void * a = realloc(nullptr, 32);       // realloc sum of func = 42
    a = realloc(a, 64);                    // realloc sum of func = 74
    a = realloc(a, 64);                    // realloc sum of func = 74
    a = realloc(a, 16);                    // realloc sum of func = 26
    free(a);
    free(b);
}

int main() {
    func();                                // realloc sum of func = 26
}
```

### Mmap mem

Sum the memory allocated by each *mmap* done at the given location.

```
void func() {
    void * a = mmap(.... 1024*1024 ....);       // mmap mem of func = 1 MB
    void * a = mremap(a, .... 2048*1024 ....);  // mmap mem of func = 2 MB
    munmap(a + 1024*1024, 1024*1024);           // mmap mem of func = 2 MB
    void * a = mremap(a, .... 2048*1024 ....);  // mmap mem of func = 3 MB
    munmap(a, 2048*1024);
}

int main() {
    func();                                     // mmap mem = 3 MB
}
```

### Mmap count

Count the number of *mmap* done at the given location.

```
void func() {
    void * a = mmap(.... 1024*1024 ....);       // mmap count of func = 1
    void * a = mremap(a, .... 2048*1024 ....);  // mmap count of func = 2
    munmap(a + 1024*1024, 1024*1024);
    void * a = mremap(a, .... 2048*1024 ....);  // mmap count of func = 3
    munmap(a, 2048*1024);
}

int main() {
    func();                                     // mmap count = 3
}
```

#### Munmap mem

Sum the memory freed by each *munmap* done at the given location.

```
void func() {
    void * a = mmap(.... 1024*1024 ....);
    void * a = mremap(a, .... 2048*1024 ....);
    munmap(a + 1024*1024, 1024*1024);           // munmap mem of func = 1 MB
    void * a = mremap(a, .... 2048*1024 ....);
    munmap(a, 2048*1024);                       // munmap mem of func = 3 MB
}

int main() {
    func();                                    // munmap mem = 3 MB
}
```

#### Munmap count

Count the number of *munmap* done at the given location.

```
void func() {
    void * a = mmap(.... 1024*1024 ....);
    void * a = mremap(a, .... 2048*1024 ....);
    munmap(a + 1024*1024, 1024*1024);           // munmap count of func = 1
    void * a = mremap(a, .... 2048*1024 ....);
    munmap(a, 2048*1024);                       // munmap count of func = 2
}

int main() {
    func();                                    // munmap count = 2
}
```

## 4.5 Informations

### 4.5.1 License

MALT is distributed under CeCILL-C license (LGPL compatible).

### 4.5.2 To cite

If you publish about MALT, you cite this research paper as reference :

```
Sébastien Valat, Andres S. Charif-Rubial, and William Jalby. 2017. MALT: a Malloc␣
↪tracker.
In Proceedings of the 4th ACM SIGPLAN International Workshop on Software Engineering for
Parallel Systems (SEPS 2017). Association for Computing Machinery, New York, NY, USA,
1-10. https://doi.org/10.1145/3141865.3141867
```

### 4.5.3 Discussion

If you have questions of remarks, you can also send me a mail at **memtt@progranet.ovh**.

## 4.6 MPI

This section will describe how to perform analysis of an **MPI application**.

### 4.6.1 Profiling an MPI application

If you want to profile an MPI application **MALT** will dump a profile for **each process** so each **rank**. But by default the files will be named with the **PID** of the process which is not what you expect in this particular case.

In order to get the rank, you first need to prepare a wrapper to replace the **PID** by the **rank**.

**Notice** it should be redone avery time you use a different MPI flavor.

```
# using mpicxx
malt --prep-mpi

# if you use another mpi C++ compiler
malt --prep-mpi my_mpi_cxx_compiler
```

Then when **launching** your **mpi** application :

```
mpirun -np 16 malt --mpi ./my_program
```

### 4.6.2 Filter do dump only some ranks

In MPI you might know that some ranks are behaving exactly the same way. In this case you can dump only a set of them :

```
mpirun -np 16 malt --mpi -o filter:ranks=0,5-8,10 ./my_program
```

## 4.7 Using custom allocator

In some cases, you application might be using a **custom memory allocator**. Here there is two cases :

1. The application is replacing the default allocator by overriding the officiel *malloc*, *free*... In this case, nothing to do as long as the replacement allocator is provided as a **dynamic library** so **malt** can be **hooked in**. 2. The applicatoin use the **standard allocator** for most of the code but use one or several **custom one** for **some parts**.

In the second case, you need to perform some dedicated actions.

### 4.7.1 Standard API with prefix

If your memory allocator natively just export the standard memory allocator API with just a **prefix** in from of all function names of the **API**, then you can simply proceed by :

Lets consider the case where you have the given correspondances :

Table 2: Table of metrics

| Custom API | Corresponding standard API |
|---|---|
| *je_malloc* | *malloc* |
| *je_calloc* | *calloc* |
| *je_free* | *free* |
| *je_realloc* | *realloc* |
| *je_memalign* | *memalign* |
| *je_posix_memalign* | *posix_memalign* |
| *je_aligned_alloc* | *aligned_alloc* |

```
# wrapping some custom allocator
malt --wrap-prefix je_

# wrapping several custom allocators
malt --wrap-prefix je_ --wrap-prefix mi
```

**Note** it will work only if the given memory allocators are build as a **shared library** outside of the application otherwise there is no ways to be **hooked in** from outside by overriding the symbols.

### 4.7.2 Non standard API

If for some reason your API does not provide exacly the list of functions listed in previous function, you need to list the wrappers one by one.

**Note** it still requires that each function follow the same API for the related symbols.

```
# wrapping only some part of the allocator because not all provided
malt --wrap malloc:je_malloc,free:je_free
```

## 4.8 Sub-part of a program

If you run on a really big program doing millions of allocation you might get a big overhead, and maybe you are just interested in a sub-part of the program. You can do it by including *malt/malt.h* in your files and use *malt_enable()* an *malt_disable()* to controle MALT on each thread. It is also a nice way to detect leaks of sub-parts of your code.

```
#include <malt/malt.h>

int main()
{
    malt_disable();
    //ignored
    malloc(16);

    malt_enable();
    //tracked
```

(continues on next page)

(continued from previous page)

```
    malloc(16);
}
```

You will need to link the *libmalt-controler.so* to get the default fake symbols when not using MALT. You can also just provide the two empty functions in your own dynamic library (not static).

If you have some allocation not under your control before your first call you can disable MALT by default on threads using the *filter:enabled* option, then enable it by hand.

If you want to change the initial status (make it disabled at start), you can simply redefine in your program the function :

```
#include <stdlib.h>
#include <malt/malt.h>

//malt will be disabled at start
int malt_init_status(void)
{
    return 0;
}

int main()
{
    //perform some costly code

    malt_enable();
    //perform some costly code you want to analyse
    malt_disable();

    //ok
    return EXIT_SUCCESS;
}
```

## 4.9 Using -finstrument-functions

MALT by default use a **backtrace** appproach to reconstruct the stacks. It has the nice advantage to perform mostly everywhere but has the problem to be very slow. This is the main source of the overhead of MALT when the app is making millions of memory allocation.

### 4.9.1 Faster instrumentation

In this case, in order to reduce the overhead you can recompile your application (ideally also the libraries otherwise they will be not seen) by using :

```
gcc -finstrument-functions -g ...
```

It will had some hooks at the entry/exit of each function so the stack is reconstruct while diffing into the call graph and not for every malloc. When making lots of mallocs it lower the overhead.

Then use malt by using :

```
malt -s enter-exit ./my_program
```

### 4.9.2 Get the stack memory usage

This option is also required to reconstruct the memory used by the stack.

**Bug**: There is currently a limitation here. Due to the way GCC implement it, the leaf function will not be measured in currentl implementation of MALT.

## 4.10 Track out of memory

When speaking about memory in HPC, one of the main issue which can arise is being killed on the computation node due to beeing **out of memory**.

MALT cannot be triggered at the exect moment of the **out of memory** being triggered. This, because at that moment the application (and the instrumentation) tool cannot do anything more than being killed.

But MALT can trigger a dump of the profile a little bit before by playing with some software threasholds so you can in post-mortem analyse the profile as usual.

### 4.10.1 Available options

You can play with the options from the **dump** group of options.

Table 3: Table of metrics

| Option | Short description |
|---|---|
| *on-signal* | Dump on signal. Can be comma separated list from SIGINT, SIGUSR1, |
| *after-seconds* | Dump after X seconds (limited to only one time) |
| *on-sys-full-at* | Dump when system memory become full at x%, xG, xM, xK, x (empty to disable). |
| *on-app-using-rss* | Dump when RSS of the app reach the given limit in %, G, M, K (empty to disable). |
| *on-app-using-virt* | Dump when Virtual Memory of the app reach limit in %, G, M, K (empty to disable). |
| *on-app-using-req* | Dump when Requested Memory of the app reach limit in %, G, M, K (empty to disable). |
| *on-thread-stack-using* | Dump when one stack reach limit in %, G, M, K (empty to disable). |
| *on-alloc-count* | Dump when number of allocations reach limit in G, M, K (empty to disable). |
| *watch-dog* | Run an active thread spying continuouly the memory of the app, not only sometimes. |

### 4.10.2 recommended approch

One recommended way if to play with *dump:on-sys-full-at* and *dump:watch-dog*. The first option permits de define a threashold when to dump base on the system memory. The good point is that is also work in a multi-process environnement.

As the system memory is looking at the **physical memory** (**rss**) consumption of the app if requires to be dynamically tracked via a **wathdog** thread as the peak can be reached between two calls of malloc which can let MALT missing it before being killed.

For the option *dump:on-sys-full-at*, select a value below *100%* as there can be side effects before reaching the max (swap…). Also MALT will use a bit a memory to dump the profile so you need some margins. By experience, **80%** looks a good value.

```
malt -o dump:watch-dog=true -o dump:on-sys-full-at=80% ./my_oom_program
```

### 4.10.3 Look in the profile

When looking in the profile, you can get the memory used a peak time with the metric **Global Peak Memory**.

## 4.11 Options

MALT supports several options which can be passed by **file** or **command line**.

The options are represented into a **two level** tree with **groups** and **options** to follow the **ini** file standard.

### 4.11.1 Command line

The **command line** options follow the simple semantic :

```
# pass the options at launch time
malt -o {GROUPE}:{OPTION}={VALUE} -o {GROUPE2}:{OPTION2}={VALUE2} ./my_program

# In a single option
malt -o "{GROUPE}:{OPTION}={VALUE};{GROUPE2}:{OPTION2}={VALUE2}" ./my_program
```

### 4.11.2 Configuration file

You can provide a **config file** to MALT to setup some features. This file uses the **INI** format. With the malt script :

```
malt -c config.ini {YOUR_PROGRAM} [OPTIONS]
```

Note you can use **file** plus override some options by adding *-o/–option* afterward :

```
malt -c config.ini -o time:points=5000 {YOUR_PROGRAM} [OPTIONS]
```

### 4.11.3 Available options

Example of config file :

```
[time]
enabled=true            ; enable time profiles
points=512              ; keep 512 points
linar-index=false       ; use action ID instead of time

[stack]
enabled=true            ; enable stack profiles
mode=backtrace          ; select stack tracing mode (backtrace|enter-exit)
resolve=true            ; Automatically resolve symbols with addr2line at exit.
libunwind=false         ; Enable of disable usage of libunwind to backtrace.
skip=4                  ; Number of stack frame to skip in order to cut at malloc level
sampling=false          ; Sample and instrument only some stack.
samplingBw=4093         ; Instrument the stack when seen passed 4K-3 bytes of alloc␣
↪requests.

[output]
name=malt-%1-%2.%3      ; base name for output, %1 = exe, %2 = PID, %3 = extension
lua=true                ; enable LUA output
json=true               ; enable json output
callgrind=true          ; enable callgrind output
```

(continues on next page)

(continued from previous page)

```
indent=false            ; indent the output profile files
config=true             ; dump current config
verbosity=default       ; malt verbosity level (silent, default, verbose)
stack-tree=false        ; store the call tree as a tree (smaller file, but need␣
↪conversion)
loop-suppress=false     ; Simplify recursive loop calls to get smaller profile file if␣
↪too big

[max-stack]
enabled=true            ; enable of disable strack size tracking (require -finstrument-
↪functions)

[distr]
alloc-size=true         ; generate distribution of allocation size
realloc-jump=true       ; generate distribution of realloc jumps

[trace]
enable=false            ; enable dumping allocation event tracing (not yet used by GUI)

[info]
hidden=false            ; try to hide possible sensible names from profile (exe, hostname.
↪..)

[filter]
exe=                    ; Only apply malt on given exe (empty for all)
childs=true             ; Instrument child processes or not
enabled=true            ; Enable or disable MALT when threads start
ranks=                  ; Instrument only the given ranks from list as : 1,2-4,6

[dump]
on-signal=              ; Dump on signal. Can be comma separated list from SIGINT,␣
↪SIGUSR1,
                        ; SIGUSR2... help, avail (limited to only one dump)
after-seconds=0         ; Dump after X seconds (limited to only one time)
on-sys-full-at=         ; Dump when system memory become full at x%, xG, xM, xK, x ␣
↪(empty to disable).
on-app-using-rss=       ; Dump when RSS of the app reach the given limit in %, G, M, K␣
↪(empty to disable).
on-app-using-virt=      ; Dump when Virtual Memory of the app reach limit in %, G, M, K␣
↪(empty to disable).
on-app-using-req=       ; Dump when Requested Memory of the app reach limit in %, G, M, K␣
↪(empty to disable).
on-thread-stack-using=  ; Dump when one stack reach limit in %, G, M, K (empty to␣
↪disable).
on-alloc-count=         ; Dump when number of allocations reach limit in G, M, K (empty␣
↪to disable).
watch-dog=false         ; Run an active thread spying continuouly the memory of the app,␣
↪not only sometimes.

[python]
instru=true             ; Enable of disable python instrumentation.
stack=enter-exit        ; Select the Python stack instrumentation mode (backtrace, enter-
```

(continues on next page)

(continued from previous page)

```
↪exit, none).
mix=false               ; Mix C stack with the python ones to get a uniq tree instread of␣
↪two distincts
                        ; (not this adds overhead).
obj=true                ; Instrument of not the OBJECT allocator domain of python.
mem=true                ; Instrument of not the MEM allocator domain of python.
raw=true                ; Instrument of not the RAW allocator domain of python.

[c]
malloc=true             ; Track C malloc.
mmap=true               ; Track C mmap direct calls.

[tools]
nm=true                 ; Enable usage of NM to find the source locatoin of the global␣
↪variables.
nmMaxSize=50M           ; Do not call nm on .so larger than 50 MB to limit the profile␣
↪dump overhead.
```

### 4.11.4 Section *time*

This set of option permits to configure the time charts about the dynamic of the application.

#### Option *time:enabled*

Enable support of tracking state values over time to build time charts.

**Default**: true.

```
malt -o time:enabled=true ./my_program
malt -o time:enabled=false ./my_program
```

#### Option *time:points*

Define the number of points used to discretized the execution time of the application.

**Default**: 512.

```
malt -o time:points=512 ./my_program
```

#### Option *time:linear_index*

Do not use time to index data but a linear value increased on each call (might be interesting not to shrink intensive allocation steps on a long program which mostly not do allocation over the run.

**Default**: false.

```
malt -o time:linear_index=true ./my_program
malt -o time:linear_index=false ./my_program
```

### 4.11.5 Section *stack*

#### Option *stack:enabled*

Enable support of stack tracking.

**Default**: true.

```
malt -o stack:enabled=true ./my_program
malt -o stack:enabled=false ./my_program
```

### Option *stack:mode*

Override by *-s* option from the command line, it set the stack tracking method to use. See *-s* documentation for more details. The available values are *enter-exit*, *backtrace*, *python*, *none*. By default backtrace is used as it works out of the box everywhere. It is slower than enter-exit but this last one needs recompilation of the code with *-finstrument-functions* and see only the libs recompiled with this option. *python* permit to project all the C allocation directly into the python stack domain so it hides the C underwood. Good to improve performance over python when you are not interested into the C details.

**Default**: backtrace.

```
malt -o stack:mode=backtrace ./my_program
malt -o stack:mode=enter-exit ./my_program
malt -o stack:mode=python ./my_program
malt -o stack:mode=none ./my_program
```

### Option *stack:resolve*

Enable symbol resolution at the end of execution to extract full names and source location if debug options is available.

**Default**: true.

```
malt -o stack:resolve=true ./my_program
malt -o stack:resolve=false ./my_program
```

### Option *stack:libunwind*

Use linunwind backtrace method instead of the one from glibc.

**Default**: false.

```
malt -o stack:libunwind=true ./my_program
malt -o stack:libunwind=false ./my_program
```

### Option *stack:skip*

In backtrace mode, the backtrace is called inside our instrumentation function itself called by malloc/free.... In order to skip our internal code we need to remove them. But on some compilers or distros, inlining and LTO configuration make wrong detection. This can be fixed by playing with this value to keep malloc/free/calloc as leaf for every calltree extracted by MALT.

If you don't see malloc as leaf, you should increase this value. Decrease if you start to see MALT internal function inside.

Notice in some cases (Gentoo, Gcc-8) we start to see LTO trashing totaly the malloc/free call in O3 mode normaly seen via backtrace. There is currently no other solution except recompiling in O0/O1 to avoid or maybe disable LTO optimizations or consider not having the exact location of the malloc itself.

**Default**: 4.

```
malt -o stack:skip=4 ./my_program
```

#### Option *stack:sampling*

Enable sampling mode for the stacks by captuing only for a few mallocs. It is less accurate than a full profile but cost less in memory and CPU. In sampling mode, the stack is processed only when we seen passed a few bytes allocated, otherwise we consider the last seen call stack. With python you should also enable the backtrace mode for solving stacks via *python:stack=backtrace*.

It will use the options *stack:samplingBw* and *stack:samplingCnt* to know at which rate to sample.

**Default**: false.

```
malt -o stack:sampling=true ./my_program
malt -o stack:sampling=false ./my_program
```

#### Option *stack:samplingBw*

Define the amount of data seen passed between two samples. Idealy this should be a prime number to avoid some multiple base biases..

It is completed by also sampling on count with *stack:samplingCnt*.

**Default**: 4093.

```
malt -o stack:samplingBw=4093 ./my_program
malt -o stack:samplingBw=5242883 ./my_program
malt -o stack:samplingBw=10485767 ./my_program
malt -o stack:samplingBw=20971529 ./my_program
```

#### Option *stack:samplingCnt*

Define the number of operations passing between two sampling. Idealy this should be a prime number to avoid some multiple base biases..

It is completed by also sampling on count with *stack:samplingBw*.

**Default**: 571.

```
malt -o stack:samplingCnt=13 ./my_program
malt -o stack:samplingCnt=31 ./my_program
malt -o stack:samplingCnt=67 ./my_program
malt -o stack:samplingCnt=67 ./my_program
malt -o stack:samplingCnt=257 ./my_program
malt -o stack:samplingCnt=571 ./my_program
```

### 4.11.6 Section *output*

#### Option *output:name*

Define the name of the profile file. %1 is replaced by the program name, %2 by the PID or MPI rank and %3 by extension.

#### Option *output:lua*

Enable output in LUA format (same structure as JSON files but in LUA).

#### Option *output:json*

Enable output of the default JSON file format.

#### Option *output:callgrind*

Enable output of the compatibility format with callgrind/kcachegrind. Cannot contain all data but can be used with compatible existing tools.

#### Option *output:indent*

Enable indentations in the JSON/LUA files. Useful for debugging but generate bigger files.

#### Option *output:config*

Dump the config INI file.

#### Option *verbosity*

Set the verbosity mode of MALT. By *default* it print at start and end. You can use *silent* mode to disable any ouptput for example if you instrument shell script parsing the output of child processes. You can also use *verbose* to have more debugging infos in case if does not work as expected, mostly at the symbol extraction step while dumping outputs.

#### Option *stack-tree*

Enable storage of the stacks as a tree inside the output file. It produces smaller files but require conversion at storage time and loading time to stay compatible with the basic expected format. You can use this option to get smaller files. In one case it lowers a 600 MB file to 200 MB to give an idea.

#### Option *output:loop-suppress*

Enable recursive loop calls to remove them and provide a more simplified equivalent call stack. It helps to reduce the size of profiles from applications using intensively this kind of call chain. In one case it lowers file from 200 MB to 85 MB. It can help if nodejs failed to load the fail because of the size. This parameter can also provide more readable stacks as you don't care to much how many times you cycle to call loops you just want to see one of them.

### 4.11.7 Section *max-stack*

#### Option *max-stack:enabled*

Enable or disable the tracking of stack size and memory used by functions on stacks (require *–finstrument-function* on your code to provide data).

### 4.11.8 Section *distr*

#### Option *distr:alloc-size*

Generate distribution of the allocated chunk size.

#### Option *distr:realloc-jump*

Generate distribution of the realloc size jumps.

### 4.11.9 Section *trace*

#### Option *trace:enabled*

Enable or disable the tracing (currently not used by the GUI, work in progress).

### 4.11.10 Section *info*

#### Option *info:enabled*

Enable hiding execution information. This option remove some possibility sensitive information from the output file, like executable names, hostname and command options. It is still recommended taking a look at the file for example to replace the paths which might also be removed. This option target some companies which might want to hide their internal applications when exchanging with external partners.

### 4.11.11 Section *filter*

#### Option *filter:exe*

Enable filtering of executable to enable MALT and ignore otherwise. By default empty value enable MALT on all executable.

#### Option *filter:childs*

Enable instrumentation of children processes or not. By default instruments all.

#### Option *filter:enabled*

Enable profiling by default. Can be disable to be able to activate via C function call in the app when you want.

#### Option *filter:ranks*

When running in MPI mode, instrument only the given ranks. The list is provided under the form *1,2-4,6*.

### 4.11.12 Section *dump*

#### Option *dump:on-signal*

Will dump on given signal. Can be on or comma separated list of: *SIGINT*, *SIGUSR1*, *SIGUSR2*, ... Notice profiling will currently stop from this point app will continue without profiling. To be fixed latter. You can get the list of availble list by using *help* or *avail* in place of name.

#### Option *dump:after-seconds*

Will dump profile after X seconds. Notice profiling will currently stop from this point app will continue without profiling. To be fixed latter.

#### Option *dump:on-sys-full-at*

Will dump if the system memory if globaly filled at x%. Use empty string to disable. Remark it consider free the free memory and the cached memory. A good value in practice is more 70% / 80% than going to 95% due to necessity to let room for the cached memory and because it will starts to swap before that. Consider also that MALT itself adds up memory on top of your one (considered in the % here.). Values can be in %, K, M, G by ending with the corresponding character.

#### Option *dump:on-app-using-rss*

Will dump if the application reach the given RSS limit. The value is given in % of the global memory available or in K, M, G. Empty to disable (default).

#### Option *dump:on-app-using-virt*

Will dump if the application reach the given virtual memory limit. The value is given in % of the global memory available or in K, M, G. Empty to disable (default).

#### Option *dump:on-app-using-req*

Will dump if the application reach the given requested memory limit. The value is given in % of the global memory available or in K, M, G. Empty to disable (default).

#### Option *dump:on-thread-stack-using*

Will dump if one of the thread stack reach the given limit. The value is given in % of the global memory available or in K, M, G. Empty to disable (default).

#### Option *dump:watch-dog*

Will start a thread which will spy the memory usage of the process and trigger the dump as soon as it sees it going to hight. This is to balance the fact than normally the system and process memory is spied only sometimes by MALT in normal condition to keep the overhead low.

### 4.11.13 Section *python*

The *python* section permit to configure how to instrument python. Notice that you need to build MALT with *–enable-python* so it is effective.

#### Option *python:instru*

Enable of disable the python instrumentation.

#### Option *python:stack*

Select the stack instrumentation mode, either *enter-exit*, *backtrace* or *none*. By default *enter-exit* is faster so you should use it. When enabling sampling you need to use *bactrace* if you don't want to pay an unneeded overhead. You can also disable python stack checking with *none*.

#### Option *python:mix*

By default when disabled the C & Python stacks are analysed independently which mean that is a python function call a C function you will see only the C call stack. Mix allow to merge the two layeres so you see that python call C. But it adds overhead on the anlysis of course because of the extra work.

#### Option *python:obj*

Analyse or ignore the object allocation domain of python. This is interesting to ignore all the small allocs used by the language as it would have been on the stack in C. It improves a lot the profiling performance of python but miss part of the memory consumption if your program store lots of small objets for long times.

#### Option *python:mem*

Same but for the mem allocation domain of python. In principle you should let it enabled.

#### Option *python:raw*

Same but for the raw allocation domain of python which is backed by the standard C malloc function. In principle you should let it enabled.

### 4.11.14 Section *c*

#### Option *c:malloc*

Track the C *malloc* calls.

#### Option *c:mmap*

Track the direct C *mmap* calls.

### 4.11.15 Section *tools*

The *tools* section permit to configure some of the sub-tools called by MALT to perform its analysis.

#### Option *tools:nm*

Use to extract the source location of the global variables. If true (default) it is used, otherwise it is skiped.

#### Option *tools:nmMaxSize*

By default it limits the size of the .so files on which to apply NM in order to keep a decent profile dumping time when running on large frameworks like PyTorch which tends to load huge .so files in memory.

## 4.12 Build options

### 4.12.1 Available options

MALT supports several build options to tune MALT at build time. Those options are listed here :

Table 4: Table of metrics

| Metric | Default | Short description |
|---|---|---|
| ENABLE_TESTS | no | Enable build of internal tests. |
| ENABLE_GCC_COVERAGE | no | Enable coverage compile flags to be enabled. |
| ENABLE_LINK_OPTIM | yes | Enable link optimisation (LTO & -fvisibility-hidden). |
| ENABLE_IPO | yes | Allow usage of IPO specfic optimization when ENABLE_LINK_OPTIM is enabled. |
| ENABLE_PYTHON | yes | Enable compiling with the python support. |
| ENABLE_JEMALLOC | yes | Enable building internal JeMalloc. |
| ENABLE_CACHING | yes | Enable some internal caches. |
| ENABLE_PREFER_EMBED | no | Prefer using the embeded dependencies that using system one by default. |
| ENABLE_CODE_TIMING | no | Enable some tricks to quickly check timings of hostspots of MALT internal code. |
| ENABLE_CODE_LEAK | no | Enable internal analysis of memory leak. |
| PORTABILITY_SPINLOCK | PTHREAD | Select spinlock implementation to use. |
| PORTABILITY_MUTEX | PTHREAD | Select mutex implementation to use. |
| PORTABILITY_OS | UNIX | Select OS implementation to use. |
| PORTABILITY_COMPILER | GNU | Select compiler support to use. |
| PORTABILITY_CLOCK | RDTSC | Clock to be used to represent time in MALT. |
| PORTABILITY_PYTHON | LAZY | Select the instrumentation mode for python. |
| WEBVIEW_SERVER | AUTO | Select the webview server to use. |

### 4.12.2 General build options

#### Option ENABLE_TESTS

Enable build of internal unit tests.

**Default**: no.

```
cmake .. -DENABLE_TESTS=no
cmake .. -DENABLE_TESTS=yes
../configure --enable-tests
../configure --disable-tests
```

#### Option ENABLE_GCC_COVERAGE

Enable coverage compile flags to be enabled.

**Default**: no.

```
cmake .. -DENABLE_GCC_COVERAGE=no
cmake .. -DENABLE_GCC_COVERAGE=yes
../configure --enable-gcc-coverage
../configure --disable-gcc-coverage
```

#### Option ENABLE_PREFER_EMBED

Prefer using the embeded dependencies that using system one by default.

**Default**: no.

```
cmake .. -DENABLE_PREFER_EMBED=no
cmake .. -DENABLE_PREFER_EMBED=yes
../configure --enable-prefer-embed
../configure --disable-prefer-embed
```

### 4.12.3 Optimisation options

#### Option ENABLE_LINK_OPTIM

Enable link optimisation (LTO & -fvisibility-hidden).

**Note**: it is not compatible with compilation of unit tests and be disabled automatically when unit tests are enabled.

**Default**: yes.

```
cmake .. -DENABLE_LINK_OPTIM=no
cmake .. -DENABLE_LINK_OPTIM=yes
../configure --enable-link-optim
../configure --disable-link-optim
```

#### Option ENABLE_IPO

Allow usage of IPO specfic optimization when ENABLE_LINK_OPTIM is enabled.

**Default**: yes.

```
cmake .. -DENABLE_IPO=no
cmake .. -DENABLE_IPO=yes
../configure --enable-ipo
../configure --disable-ipo
```

### 4.12.4 Feature options

#### Option ENABLE_PYTHON

Enable compiling with the python support. In order to be enabled, it requires the presence the the **Python C header files**.

**Default**: yes.

```
cmake .. -DENABLE_PYTHON=no
cmake .. -DENABLE_PYTHON=yes
../configure --enable-python
../configure --disable-python
```

#### Option ENABLE_JEMALLOC

Enable building internal JeMalloc.

It permits to boost MALT and reduce its internal memory overhead by being more efficient than the lightweight fallback internal memory allocator.

**Default**: yes.

```
cmake .. -DENABLE_JEMALLOC=no
cmake .. -DENABLE_JEMALLOC=yes
../configure --enable-jemalloc
../configure --disable-jemalloc
```

#### Option ENABLE_CACHING

Enable some internal caches to boost the performance of MALT and reduce its overhead.

You can get some statistics about the number of miss / hits by using the ENABLE_CODE_TIMING option.

**Default**: yes.

```
cmake .. -DENABLE_CACHING=no
cmake .. -DENABLE_CACHING=yes
../configure --enable-caching
../configure --disable-caching
```

### 4.12.5 Developer options

#### Option ENABLE_CODE_TIMING

Enable some tricks to quickly check timings of hostspots of MALT internal code. The **metrics** will be printed **at the exit** of MALT in the **terminal**.

**Default**: no

```
cmake .. -DENABLE_CODE_TIMING=no
cmake .. -DENABLE_CODE_TIMING=yes
```

#### Option ENABLE_CODE_LEAK

Enable some internal analysis of the MALT core to detect internal memory leaks in the tool itself.

**Default**: no

```
cmake .. -DENABLE_CODE_LEAK=no
cmake .. -DENABLE_CODE_LEAK=yes
```

### 4.12.6 Portability options

Those option are there to ease the portability of MALT and to adapt it to the system in use.

#### Option PORTABILITY_SPINLOCK

Select spinlock implementation to use. Currently support : **PTHREAD**, **DUMMY**.

**Default**: PTHREAD

```
cmake .. -DPORTABILITY_SPINLOCK=PTHREAD
cmake .. -DPORTABILITY_SPINLOCK=DUMMY
```

### Option PORTABILITY_MUTEX

Select mutex implementation to use. Currently support : **PTHREAD**, **DUMMY**.

**Default**: PTHREAD

```
cmake .. -DPORTABILITY_MUTEX=PTHREAD
cmake .. -DPORTABILITY_MUTEX=DUMMY
```

### Option PORTABILITY_OS

Select OS implementation to use. Currently support : **UNIX**.

**Default**: UNIX

```
cmake .. -DPORTABILITY_OS=UNIX
```

### Option PORTABILITY_COMPILER

Select compiler support to use. Currently support : **GNU** (also valid intel compiler toolchain).

**Default**: GNU

```
cmake .. -DPORTABILITY_COMPILER=GNU
```

### Option PORTABILITY_CLOCK

Clock to be used to represent time in MALT.

**Default**: RDTSC

```
cmake .. -DPORTABILITY_CLOCK=RDTSC
```

### Option PORTABILITY_PYTHON

elect the instrumentation mode for python : **LAZY**, **NATIVE**.

**Default**: LAZY

```
cmake .. -DPORTABILITY_PYTHON=LAZY
cmake .. -DPORTABILITY_PYTHON=NATIVE
```

### Option WEBVIEW_SERVER

Select the webview server implementation to use : **AUTO**, **CPP**, **NODEJS**, **RUST**, **OFF**.

**Default**: AUTO

```
cmake .. -DWEBVIEW_SERVER=AUTO
cmake .. -DWEBVIEW_SERVER=CPP
cmake .. -DWEBVIEW_SERVER=NODEJS
cmake .. -DWEBVIEW_SERVER=OFF
# experimental, not yet production ready
cmake .. -DWEBVIEW_SERVER=RUST
```

## 4.13 Packaging for distributions

MALT provide the necessary script to generate the packages for common distributions. You will find here the procedure to follow to build them.

### 4.13.1 Archive from the git sources

Before following the rest of this document you need either to:

- Download an **official release** tarball.
- **Generate** the release tarball.

To generate from the **git sources** :

```
./dev/dev.py archive --commit v1.4.0 --version 1.4.0 --no-hash
```

### 4.13.2 Gentoo

On Gentoo you can directly use the Overlay provided on the web : https://github.com/memtt/gentoo-memtt-overlay

The usage procedure is given here :

```
sudo eselect repository add memtt git https://github.com/memtt/gentoo-memtt-overlay.git
sudo eselect repository enable memtt
sudo emerge -a malt
```

### 4.13.3 RPM : Fedora and others

If you want to build your package for Fedora or related RPM based distribution, you can follow the given commands.

```
# Extract the packging dir if you are not in git sources
tar --strip-components 1 -xvf malt-1.4.0.tar.bz2 malt-1.4.0/packaging/fedora

# build the podman image
podman build -t malt/fedora-rpmbuild -f packaging/fedora/Dockerfile

# Call the script inside docker
podman run --rm -ti -v .:/sources:rw malt/fedora-rpmbuild /sources/packaging/fedora/
↪build.sh 1.4.0
```

### 4.13.4 APT : Debian and others

If you want to build your package for Debian, Ubuntu or related APT based distributions, you can follow the given commands.

```
# Extract the packging dir if you are not in git sources
tar --strip-components 1 -xvf malt-1.4.0.tar.bz2 malt-1.4.0/packaging/debian

# build the podman image
podman build -t malt/debian-debbuild -f packaging/debian/Dockerfile

# Call the script inside docker
podman run --rm -ti -v .:/sources:rw malt/debian-debbuild /sources/packaging/debian/
↪build.sh 1.4.0
```

## 4.14 Other tools

Of course MALT is not the only tool available to profile the memory behavior of an application.

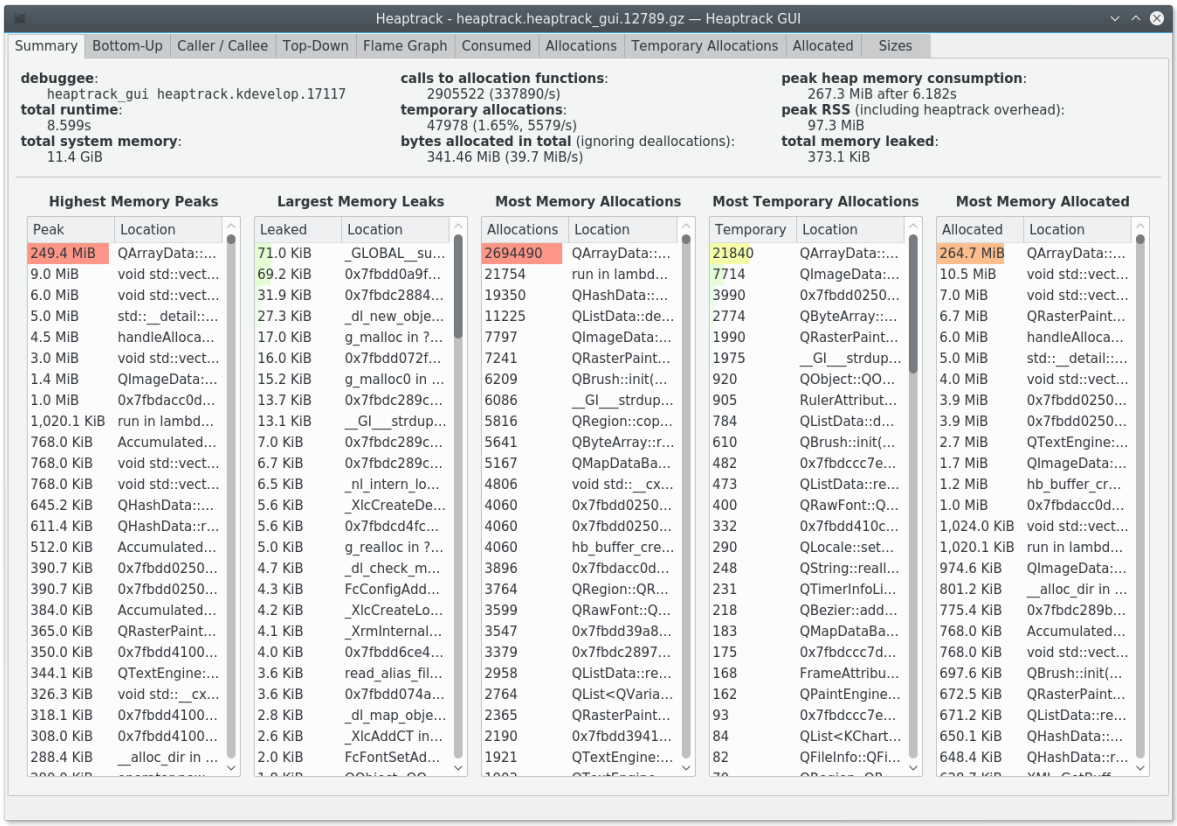
Here a list (certainly incomplete) of the others I know.

### 4.14.1 Heaptrack

A Heap Memory Profiler for Linux from the KDE team. In some ways it has some close idea to MALT.

This one has the advantage to be **officially packaged** in distributions like Debian / Ubuntu.

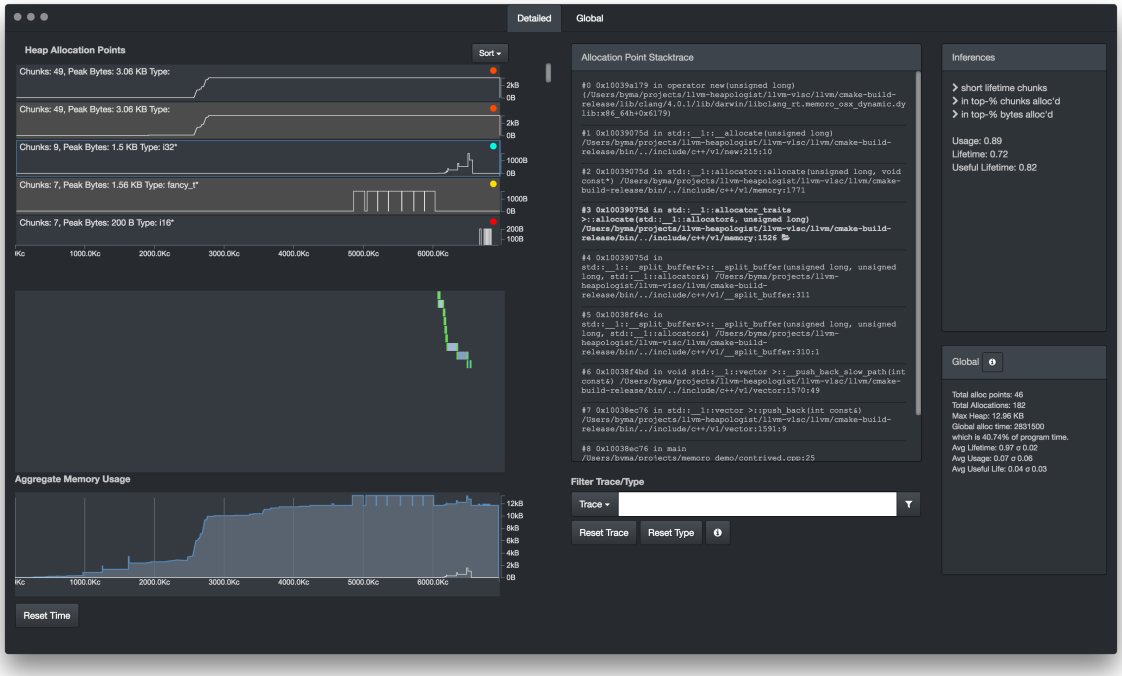
**URL**: https://github.com/KDE/heaptrack




### 4.14.2 Memoro

A memory profiler close to MALT also in some ways, for the dynamic part of the memory management.

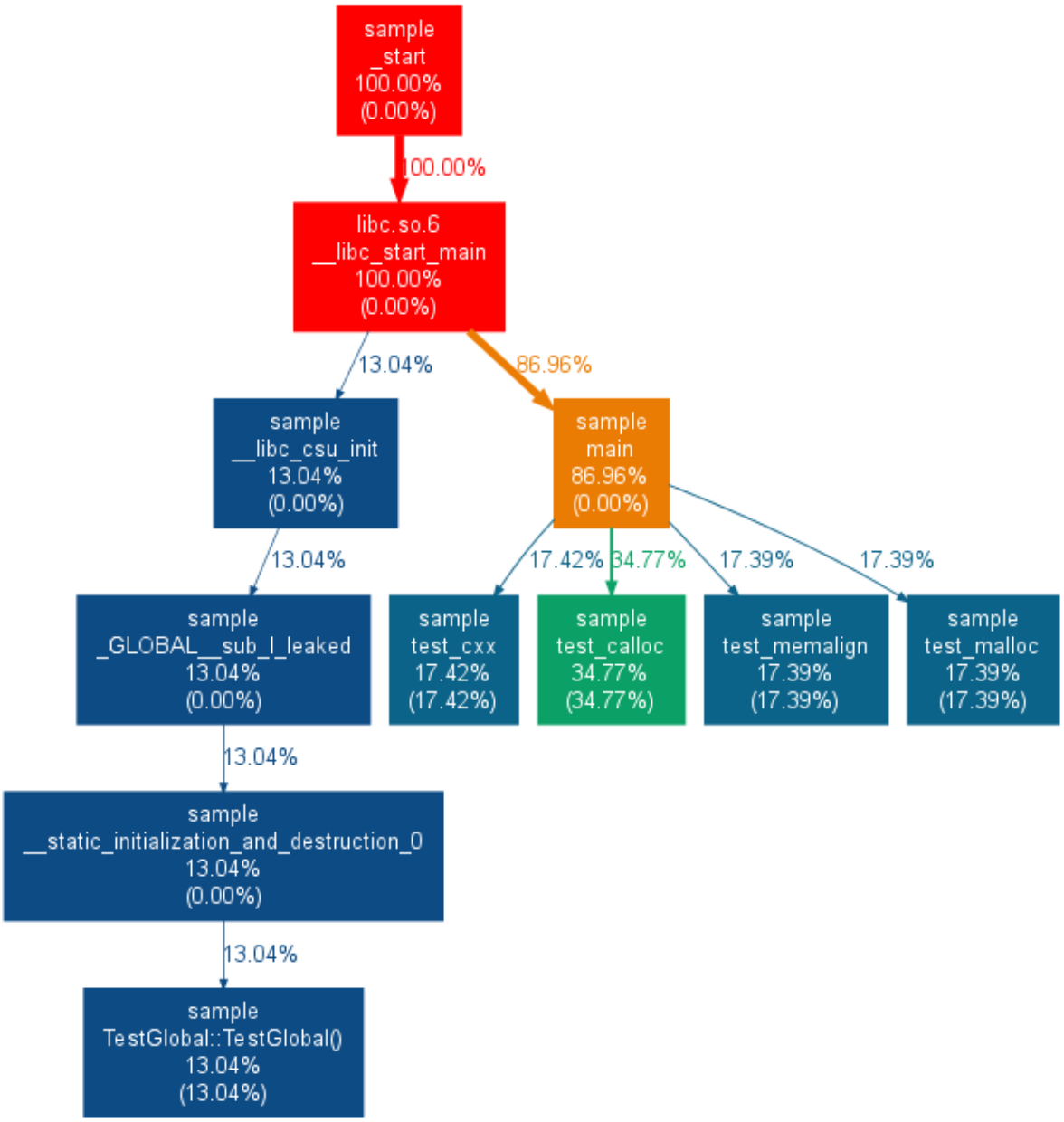
**URL**: https://epfl-vlsc.github.io/memoro/




### 4.14.3 Memtrail

A tool to report the callocations on the call tree just like the tree part of MALT but in a static way.

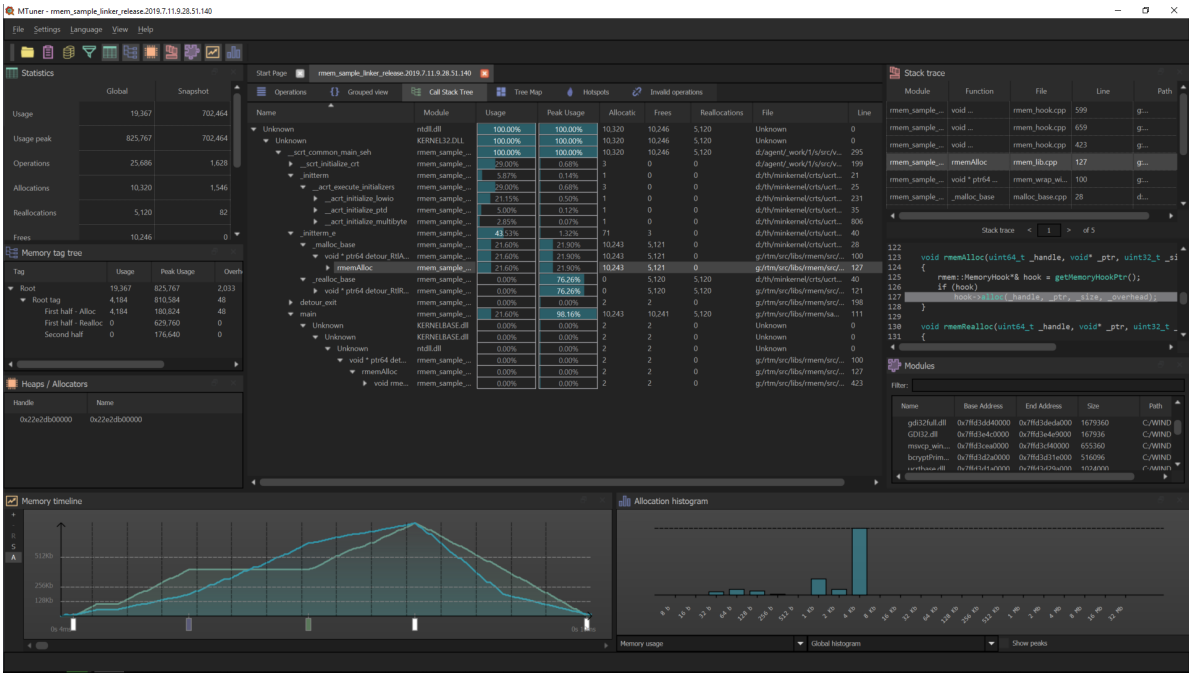
**URL**: https://github.com/jrfonseca/memtrail




### 4.14.4 MTuner

Another memory profiler, in some ways close to MALT.

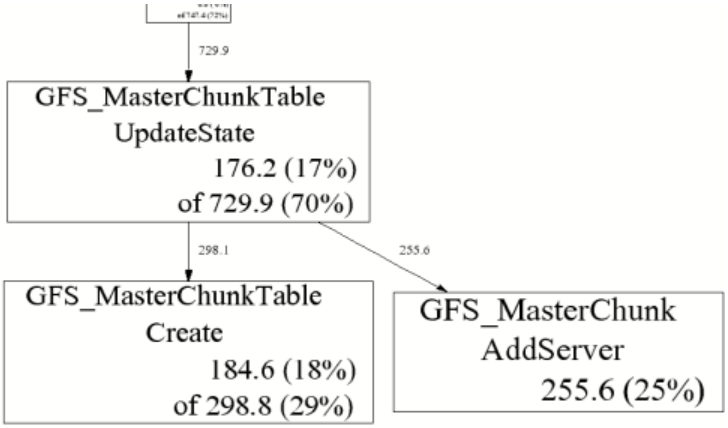
**URL**: https://github.com/RudjiGames/MTuner



### 4.14.5 Google Heap Profiler

Google heap profiler is a light memory profiler comming with **TCMalloc**, the memory allocator from google. It permits to annotate the call graph with some memory metrics.

**URL**: https://gperftools.github.io/gperftools/heapprofile.html




### 4.14.6 Valgrind memcheck

Valgrind is a well know tool to perform analysis of programs. One of its sub tools is **memcheck** which permits to detect the wrong memory accesses and the memory leaks in a compiled program. The output is pure texte in the terminal.

**URL**: http://valgrind.org/

```
==19182== Invalid write of size 4
==19182==    at 0x804838F: f (example.c:6)
==19182==    by 0x80483AB: main (example.c:11)
==19182==  Address 0x1BA45050 is 0 bytes after a block of size 40 alloc'd
```

(continues on next page)

(continued from previous page)

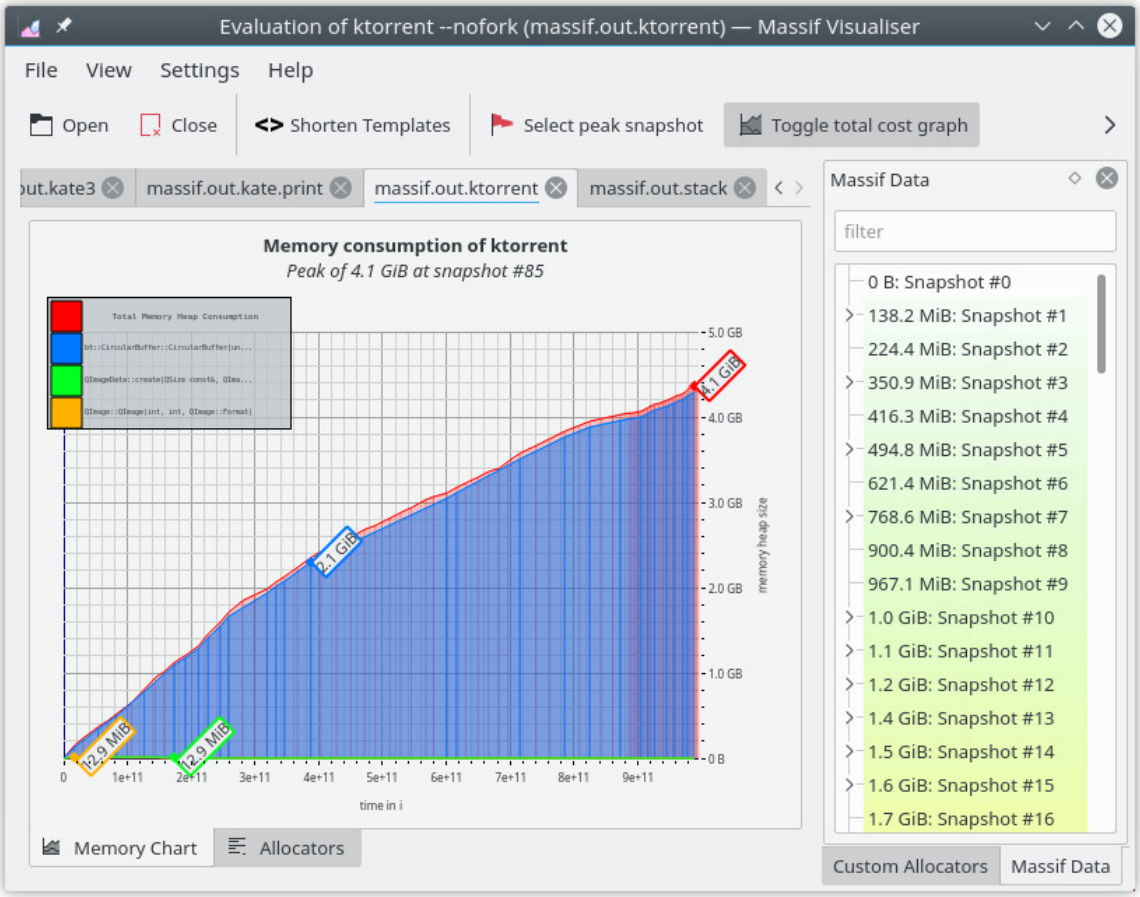
```
==19182==    at 0x1B8FF5CD: malloc (vg_replace_malloc.c:130)
==19182==    by 0x8048385: f (example.c:5)
==19182==    by 0x80483AB: main (example.c:11)
```

### 4.14.7 Valgrind massif

Valgrind is a well know tool to perform analysis of programs. One of its sub tools is **massif** which aims at giving hints about memory consumption of a program. It comes with a KDE GUI (**massif visualizer**) to display the profile.

**URLs:**

- http://valgrind.org/
- https://apps.kde.org/fr/massif_visualizer/




### 4.14.8 Dr. Memory

Similar to Valgrind Memcheck it search for memory access issues.

```
~~Dr.M~~ ERRORS FOUND:
~~Dr.M~~       5 unique,     5 total,    574 byte(s) of leak(s)
~~Dr.M~~       0 unique,     0 total,      0 byte(s) of possible leak(s)
~~Dr.M~~ ERRORS IGNORED:
```

(continues on next page)

(continued from previous page)

```
~~Dr.M~~       5 unique,     8 total,    205 byte(s) of still-reachable allocation(s)
~~Dr.M~~          (re-run with "-show_reachable" for details)
```

### 4.14.9 Unicom Purify++

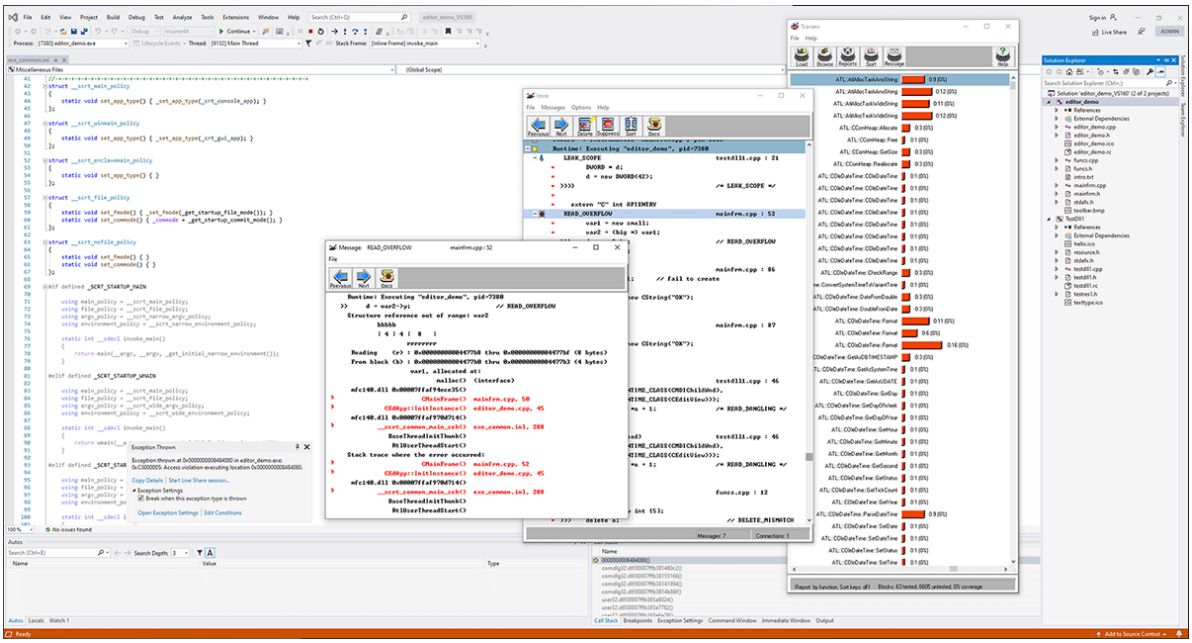
This one is not open-source but commercial and available on Windows, Linux, Solaris. It provides lots of metrics about memory managment.

**URL**: https://www.unicomsi.com/products/purifyplus/

### 4.14.10 Parasoft Insure++

This one is not open-source but commercial and only for Windows. It provides lots of metrics about memory management.

**URL**: https://www.parasoft.com/product/insure/



### 4.14.11 Tau

**Tau** is a well known HPC large scale profiler for super-computers. As a complete tool it also contains some modules about memory usage.

**URL**: https://www.cs.uoregon.edu/research/tau/home.php

```
USER EVENTS Profile :NODE 0, CONTEXT 0, THREAD 0
---------------------------------------------------------------------------------------
NumSamples   MaxValue   MinValue  MeanValue  Std. Dev.  Event Name
---------------------------------------------------------------------------------------
         2          52          48          50          2  MEMORY LEAK! malloc size
↪<file=simple.inst.cpp, line=18> : int g(int)   => int bar(int)
         1          80          80          80          0  free size <file=simple.inst.cpp,␣
↪line=21>
         1          80          80          80          0  free size <file=simple.inst.cpp,␣
```

(continues on next page)

(continued from previous page)

```
↪line=21> : int g(int)   => int bar(int)
         1        180        180        180          0  free size <file=simple.inst.cpp,␣
↪line=28>
         1        180        180        180          0  free size <file=simple.inst.cpp,␣
↪line=28> : int foo(int)   => int bar(int)
         3         80         48         60      14.24  malloc size <file=simple.inst.
↪cpp, line=18>
         3         80         48         60      14.24  malloc size <file=simple.inst.
↪cpp, line=18> : int g(int)   => int bar(int)
         1        180        180        180          0  malloc size <file=simple.inst.
↪cpp, line=26>
         1        180        180        180          0  malloc size <file=simple.inst.
↪cpp, line=26> : int foo(int)   => int bar(int)
-------------------------------------------------------------------------------------
```

### 4.14.12 IgProf

Similar approach than MALT for the backend.

**URL**: http://igprof.org/

### 4.14.13 Dmalloc

A debug malloc library.

**URL**: https://dmalloc.com/

### 4.14.14 mpatrol

**URL**: https://mpatrol.sourceforge.net/

### 4.14.15 FOM

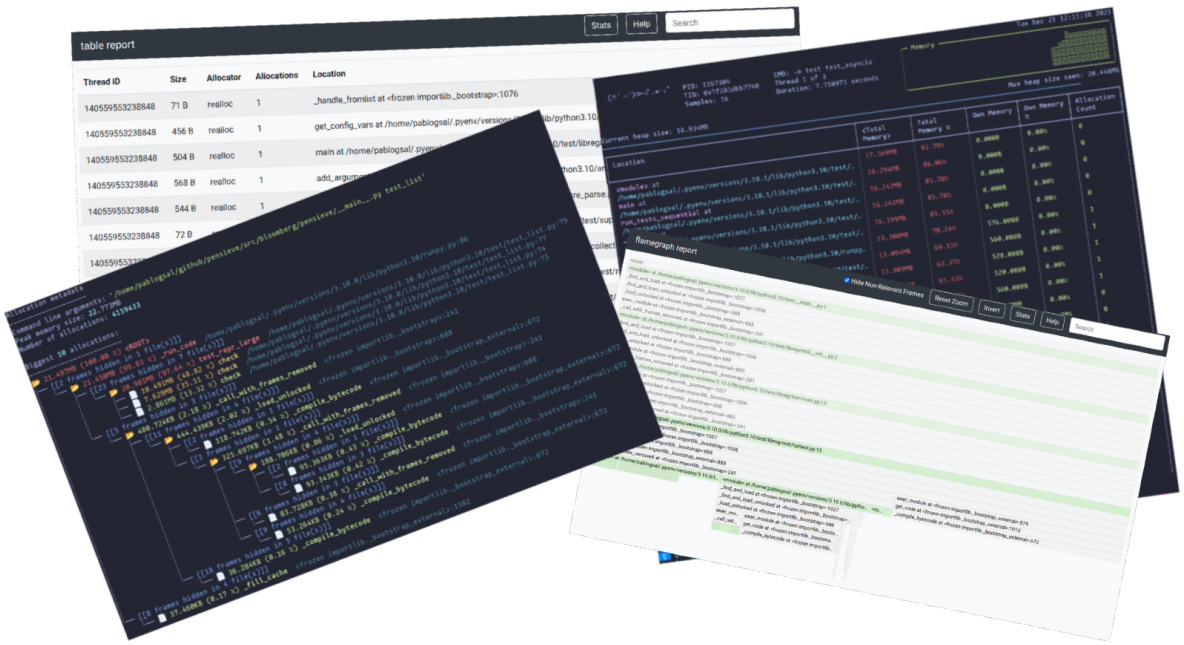
Find Obsolete Memory.

**URL**: https://github.com/FOM-Tools/FOM-Tools

### 4.14.16 Memray

Dedicated for Python, this tool is well done to understand the memory behavior of a Python code.

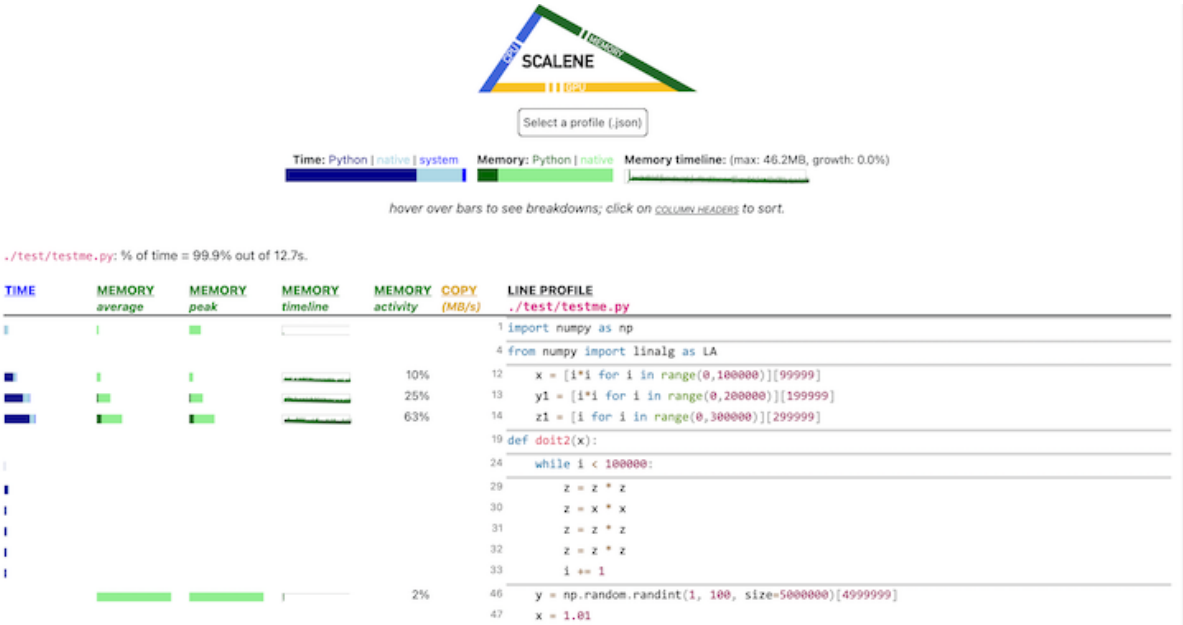
**URL**: https://bloomberg.github.io/memray/




### 4.14.17 Scalene

Tool to analyse the performance an memory behavior of a code in C / Python.

**URL**: https://pypi.org/project/scalene/




## 4.15 Other memory allocators

There is several very well done memory allocator implementations available in the open-source community. As sometimes it can offer very nice performance or memory consumption improvement in application, it is interesting to test them.

The one I know that behave very nicely depending on the patterns of the app are listed here.

### 4.15.1 How to use a custom allocator

If your application is built in a non static way, the easier way to try another memory allocator is just to compile it and inject in in the allocation using :

```
LD_PRELOAD=libjemalloc.so ./my_program
LD_PRELOAD=libtcmalloc.so ./my_program
#...
```

### 4.15.2 JeMalloc

Implemented by John Evans, a very nice memory allocator, mostly optimal to reduce the memory consumption of an applicaton. It has very nice algorithms to manage the small blocks in an efficient way. It is also natively done for parallel codes.

When allocating lors of large segment, it tends to exchange them oftenly with the system which for HPC application tend to increase the execution time.

I studied it a lot during my PhD. about memory managment and looks to me to have a greater potential.

**URL**: https://jemalloc.net/

### 4.15.3 TCMalloc

A memory allocator implemented by Google and very nicely done to take in account the cache effects about memory management. It oftenly offer very nice performance but at the price of the memory consumption depening on the allocation pattern of your code. It is also natively done for parallel codes.

**URL**: https://github.com/gperftools/gperftools

### 4.15.4 Hoard

The memory allocator written by Emery Berger, a well known researcher about memory management. This one has nice featur about parallelism, and is historically a well inspirator about the ways to handle parallelism at the allocator level.

**URL**: http://www.hoard.org/

### 4.15.5 Lockless allocator

Another memory allocator done for parallelism.

**URL**: http://locklessinc.com/downloads/

### 4.15.6 MPC Allocator

A memory allocator developped for the MPC (Multi-Processor Computing framework). This is inspired about the work of the previous allocators and implemented by the core author of **MALT**. It is secifically developped form NUMA architectures and permits to switch between a low memory profile (slower) to a faster running mode (hight memory consumption). This one is specifically tuned to handle well the large memory allocations done in HPC applications.

**URL**: https://github.com/cea-hpc/mpc-allocator

### 4.15.7 Mimalloc

A memory allocator we tend to see in many posts about Python community done by Microsoft.

**URL**: https://github.com/microsoft/mimalloc

## 4.16 Documentation about memory

If you want to better understand things about memory managment you can probably be interested by the following documents :

### 4.16.1 What Every Programer Should Now About Memory

A very nice document written by **Urlich Drepper** in 2007, starting from the silicon to the application level.

**URL**: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

### 4.16.2 Phd. thesis about memory managment in HPC

I made my PhD. about memory managment in HPC, you can find the final thesis (**in French**, sorry). It contains an introduction on the memory managment stack.

**URL**: https://hal.archives-ouvertes.fr/tel-01253537/file/2014-valat-pdh-theses.pdf